Aderant

BETTER TOGETHER

Expert Back Office powered by CMS.NET

Common Elements API

Reference Guide (January 2018)

This document contains proprietary and confidential information of Aderant Holdings, Inc. ("Aderant")

Aderant makes no representations or warranties with respect to the contents of this document publication and specifically disclaims any implied warranties of merchantability or fitness for any particular purpose.

The contents of this manual are believed to be current and accurate as of its date of publication. Changes to this manual between reprintings and other important information about the software product are made or published in release notes, and you are urged to obtain the current release notes for the software product.

We welcome user comments and reserve the right to revise this publication and/or make improvements or changes to the products or programs described in this publication at any time, without notice.

Aderant

BETTER TOGETHER

Copyright © 2018, Aderant Holdings, Inc. All rights reserved.

Aderant, Aderant Expert, and the Aderant logo are registered trademarks or trademarks of Aderant Holdings, Inc. Other brand and product names are trademarks or registered trademarks of their respective owners, and may include trademarks owned by subsidiaries and affiliated entities of Aderant Holdings, Inc.

Printed in the United States of America. No part of this publication may be reproduced in any form without the prior written consent of Aderant Holdings, Inc.

Aderant North America

The Pointe

500 Northridge Road, Suite 800

Atlanta, Georgia 30350

Tel: +1.404.720.3600

Fax: +1.404.720.3601

info@aderant.com | www.aderant.com

Table of Contents

Introduction **1**
About this API Reference 1
Objectives of This Guide 1
Scope and Limitations of This Reference 1
Intended Audience 1
Required Proficiencies 2
Disclaimer 2
Primer **3**
Overview 3
Business Logic Components (BLCs) 3
Data Classes 4
Affecting Data 4
Retrieving Data 4
Database Locking 5
Security 5
Execution Environments 6
Visual Components (VCs) 6
Security 7
Parameters 7
Entity Filter Expressions 7
DAC Filters 7
Visual Component List Filters 7
Expression Syntax 8
Consuming Visual Components **10**
Execution Environments 10
Portals 10
Email Systems 10
Web Applications 11
Browser Control Environment 12
Considerations 12
Host Environment 12
Authentication 12

Consuming BLCs **13**
Execution Interfaces 13
.NET Interface 13
Web Services Interface 13
Extending Functionality 14
BLC Events 14
Considerations 14
Host Environment 14
Authentication 15
OfficeXtension 15
Getting Started 16
Example 1 16
Example 2 18
Example 3 19
Example 4 20
API Reference **22**
Authentication 22
Authentication/AuthenticationService.asmx 22
CioPrincipal 24
ICioPrincipal 24
BOS Request Manager 25
RequestManager/BOSService.asmx 25
Database Locking 29
DbLock/DbLockService.asmx 29
OwnerlessDbLock/OwnerlessDbLockService.asmx 31
Document Generation BLC 32
BOS Parameters 32
Example 33
User Rights 33
FunctionalSecurityService.asmx 33
Getting Support **34**

Introduction

About this API Reference

Customizability has always been a cornerstone of Expert Back Office applications. In order to assist our clients in their customization efforts, ADERANT makes available for separate purchase the Programming Interface Reference set, which consists of this document (Browser Common Elements API Reference) and the following application-specific references:

- Browser File Opening API Reference
- Browser Inquiries API Reference
- Browser Billing API Reference
- Browser Time API Reference
- Browser Conflicts API Reference
- Browser Expenses API Reference

The Common Elements API Reference addresses those elements of the Browser API that are used/consumed by multiple applications. This API Reference contains the following sections:

1. **Primer**. Provides an overview of the Application Programming Interface, along with information on Business Logic Components, Business Object Server and Visual Components.
2. **Consumption.** These sections provide information on how to consume both visual components and business logic components.
3. **API Reference**. Information on the common elements comprising the Browser API. These include: Authentication, BOS Request Manager, Database Locking, Document Generation, and User Rights.

ADERANT grants use of the contents of these application programming interfaces to clients who receive this material directly from ADERANT, and prohibits any use or access to this material by third parties.

Objectives of This Guide

The Browser Application Programming Interface (API) Reference is intended to supply building blocks that, along with the specific application API references, allow client developers to create custom applications, automate processes, and vary deployment models.

Scope and Limitations of This Reference

This Reference will provide information and instruction on common elements of the Browser API. While this document provides some information relating to the APIs of specific ADERANT Expert Browser applications and their functions, it is not intended as a guide to those APIs. See the appropriate Browser application API for details on that application.

Intended Audience

The Browser API Reference serves the needs of client developers wishing to extend, customize, or modify ADERANT Expert functionality and its deployment. Developers working with this API must be familiar with the Expert Back Office CS (client/server) database.

Required Proficiencies

The building blocks for this API are visual components and business logic components (BLCs). Each BLC interface requires different proficiencies.

Visual Components

There are no special proficiencies for working with visual components.

Business Logic Components

.NET Interface

Developers who wish to work with .NET classes directly must be proficient with the .NET Framework and have a basic understanding of COM+. All .NET examples given in this documentation are in C#, but other .NET languages may be used to access the API.

Web Service Interface

Developers who wish to call web services must be proficient with Simple Object Access Protocol (SOAP) or with the web service interface that applies to your development environment.

Those using the API through the web service can work with SOAP directly, or use a tool provided by their development environment of choice that encapsulates the complexity of working with SOAP into an object or set of functions available through the environment. For example, the .NET Framework provides a set of classes for working with ADERANT Expert system services, and Visual Basic 6 and other COM-capable environments can use system services via the Microsoft SOAP Toolkit 2.0.

BOS Execution

Some BLCs are executed on a Business Object Server (BOS), instead of the web server. These BLCs are invoked indirectly through the BOS Request Manager API and therefore require proficiencies as discussed in the .NET or Web Service interface sections above. Additionally, developers who wish to work with BOS-based BLCs must be proficient with extensible markup language (XML). XML is required to correctly format parameters when submitting a BOS request.

Disclaimer

The code samples and other examples provided in this ADERANT document are provided “as is” without warranty of any kind. ADERANT cautions **recipient** to determine for self the suitability of the use of the code samples and other data provided as examples of functionality in this document for any purpose. To the extent permitted by law or statute, ADERANT disclaims all warranties, either expressed or implied, including the warranties of merchantability and fitness for a particular purpose. To the extent permitted by law or statute in no event shall ADERANT or its suppliers be liable for any damages whatsoever including direct, indirect, incidental, consequential, loss of business profits or special damages, even if ADERANT or its suppliers have been advised of the possibility of such damages.

Primer

Overview

The Application Programming Interface (API) Reference provided here supplies building blocks that allow client developers to create custom ADERANT Expert applications, automate processes, and vary deployment models. ADERANT grants use of the content herein to clients who receive this material directly from ADERANT and prohibits any use or access to this material by third parties.

Developers can use two types of building blocks to create custom applications or to customize the deployment of Expert Back Office functionality:

- Business Logic Components
- Visual Components

Business Logic Components (BLCs) provide the underlying functionality of the Expert Back Office applications. A single service call can include a prebill selection transaction or bill drafting transaction with the DocuDraft engine. In this way, developers can build their own web or desktop-based solutions that access BLCs. BLC functionality can be accessed using either a native .NET interface or a SOAP-based web services interface. Each of these platforms has advantages and disadvantages that might make it more appropriate for a specific use.

Visual components (VCs) are web pages that usually appear in Expert Back Office applications in an upper or lower HTML frame. With the release of visual components to client developers, clients can now incorporate separate pieces of Expert Back Office functionality in web pages, in Microsoft (MS) Outlook, in MS SharePoint or in custom applications. In order to deploy visual components in custom applications, developers must place them in a Microsoft **Internet Explorer 6** browser control. Earlier versions of Internet Explorer (IE) are not compatible with Expert Back Office visual components. VCs are accessed using a URL-based API in which query parameters are used to control the behavior of the component.

Business Logic Components (BLCs)

Business Logic Components (BLCs) provide the underlying functionality of the ADERANT Expert applications. BLCs are divided into two types: those used to **retrieve** data and those used to **affect** it. Those that retrieve data, providing read-only access to the data, are more specifically referred to as Data Access Components (or DACs).

All .NET BLCs are supported by one data class and typically two derivative classes (some BLCs may have additional derivative data classes). These classes are used to buffer data being returned from DAC methods as well as being passed to BLC methods.

BLC methods can be executed through either of the supported API platforms: .NET components or web services. Each has its advantages and disadvantages that might make it more appropriate for a specific implementation. In addition, each API platform requires different knowledge prerequisites. Using .NET components, for example, require knowledge of the .NET Framework and how COM+ transactions work.

Data Classes

Data is passed into and out of the .NET BLCs using data classes. These classes contain members that represent all of the parameters into a function that manipulate data, or all the data that can be returned from the database.

To retrieve the data required, each .NET BLC works with a data class of the same prefix. For instance, the **AddressBLC** uses the **AddressData** class for data retrieval. Each BLC has one data class and may have up to two derivative classes accessed via functions. For example, **AddressData** supplies functions that access two supporting classes: **AddressDataCreate** and **AddressDataUpdate**.

Data classes begin with the name of the BLC/DAC they serve (e.g., Address), and end with one of the following suffixes:

- **Data:** Contains all fields that can be read from the database using a particular DAC.
- **DataCreate:** Contains only the fields that can be modified using the **Create** method on the BLC.
- **DataUpdate:** Contains only the fields that can be modified using the **Update** method on the BLC.

In cases where a **DataCreate** or **DataUpdate** class is available, the **Data** class contains methods for obtaining these objects with their values defaulted from the **Data** class that originated the object.

Affecting Data

Business Logic Components (those used to affect data) generally publish three methods to manipulate data in the database:

- **Create:** Creates a new row. In most cases, the **Create** method returns the primary key of the newly created item. This method takes a data class ending in **DataCreate**.
- **Update:** Updates information in the database. This method takes a data class ending in **DataUpdate**.
- **Delete:** Removes information from the database. This method usually takes the primary key value of the row to be removed. This method sometimes takes a Boolean parameter indicating whether "child" rows also should be deleted.

Retrieving Data

.NET BLCs that simply retrieve data are called Data Access Components (DACs). DACs generally publish three methods:

- **Initialize:** Returns a **Data** object with default values. The caller can use the **GetDataCreate** method to get an object with initialized values that can then be passed to a BLC's **Create** method.
- **ReadSingle:** Retrieves a single record based on its primary key.
- **Read:** Returns an array of data objects representing the rows in the database that meet the filter passed in. The *filter* parameter is a standard entity filter expression based on the entity named within the Reference section.

The Getting Started topic provides sample BLC code. See Entity Filter Expressions later in the primer for information on formulating entity filter expressions.

Database Locking

Database locking ensures database integrity by denying access to individual rows within the Back Office for Client/Server database during a specific operation, i.e., a BLC service. For instance, during editing a time entry, the Time Entry BLC will confirm that the record being modified is indeed locked by the caller. This prevents other users from accessing the row and concurrently saving data.

The application-specific API reference documents any required locks enforced by the various BLC methods.

Establishing a Lock

Typically, the caller of a BLC method is responsible for acquiring any required locks (using the DbLock .NET BLC or DbLockService web service) and releasing them when no longer needed. All owned locks last for a set duration (default is 30 minutes) from the point in which they are created. After this period, they are automatically released by the system. If a required DbLock is not currently held, a BLC method will throw an exception.

Security

User rights secure functional capabilities of a BLC throughout the firm. Beyond being authenticated by CMS.Net, users must also be granted rights to execute DAC methods as well as the specific methods of the transactional BLCs. The System Administrator uses the CMS.Net for Client/Server Security application to determine:

- Which tasks (functional capabilities) the user can perform within a BLC
- Whether row security is disabled within a BLC

To execute any method of a DAC, the user must have **execute** rights on the module representing the "owning" browser application. For example, the Activity DAC is owned by the browser time application, so a user would need the following right:

Module	Rights Name
CMS.Net Time Entry Application 1,50000,50700	Execute

Each transactional BLC contains its own entry in the security module tree. A specific rights flag typically secures each method so that access is not an "all or nothing" decision. In this way, each functional capability (e.g., Print, Edit, Post) within a BLC is granted or denied to individual users and user groups.

For example, the time entry BLC supports the following rights:

Module	Rights Name
CMS.Net Time Entry Application/CMS.Net Time Entry Business Logic Components/CMS.Net Time Entry BLC 1,50000,50700,50702,50703	Execute Create Edit Delete Release

Accessing a BLC method without user rights will result in an exception being thrown. If desired, developers can use the API to check the user's rights before invoking a BLC method. See the Authentication and User Rights topics in the Reference section for more information.

Execution Environments

Most BLC methods are executed locally on the machine containing the .NET assembly that contains the implementation of the component. These assemblies typically reside on web servers hosting the application. This provides good performance for the web servers that call them frequently in serving requests to web pages and web services. BLC methods that are substantially more CPU intensive than others are instead executed on a special dedicated machine called a Business Object Server (BOS). Any number of BOS machines can be configured to assist with the processing burden of these methods.

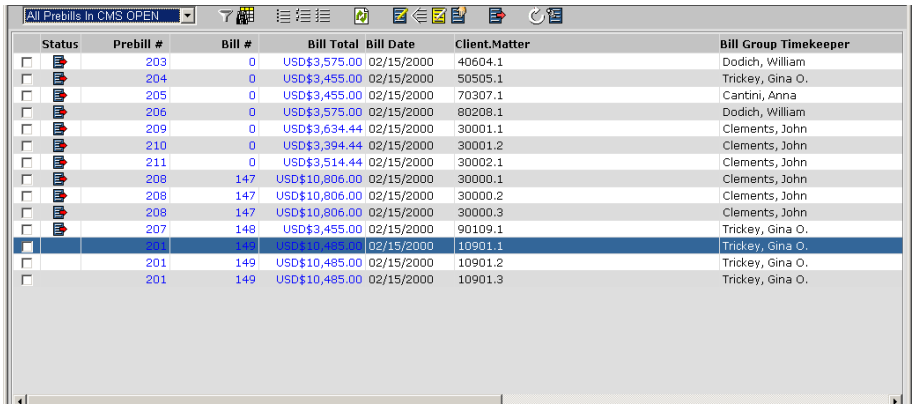
BOS Requests

To invoke a BLC method that requires a business object server to execute it, a BOS request needs to be issued. Once the request is complete, an eligible BOS server will pick up the request and process the job remotely. This is accomplished using the BOS Request Manager (through either the .NET component or web services interface).

Each BOS request includes one or more parameters. The parameters for methods executed by BOS are expressed in XML. This permits them to be easily stored in the database where the request queue is maintained. The application-specific API Reference documents provide a list of parameters for each BLC method that is executed through BOS.

Visual Components (VCs)

Visual components (VCs) are the basic building blocks for all CMS.Net applications. They provide developers with a visual API that accesses the discrete visual functionality found within CMS.Net applications for incorporation into custom solutions and deployments. Developers can incorporate visual components in web pages, Microsoft (MS) Outlook, MS SharePoint and other portal environments. Visual components are reliant on the Internet Explorer 6.0 (IE) browser control and as such can be utilized by any environment that can host or launch the control.



Status	Prebill #	Bill #	Bill Total	Bill Date	Client.Matter	Bill Group Timekeeper
	203	0	USD$3,575.00	02/15/2000	40604.1	Dodich, William
	204	0	USD$3,455.00	02/15/2000	50505.1	Trickey, Gina O.
	205	0	USD$3,455.00	02/15/2000	70307.1	Cantini, Anna
	206	0	USD$3,575.00	02/15/2000	80208.1	Dodich, William
	209	0	USD$3,634.44	02/15/2000	30001.1	Clements, John
	210	0	USD$3,394.44	02/15/2000	30001.2	Clements, John
	211	0	USD$3,514.44	02/15/2000	30002.1	Clements, John
	208	147	USD$10,806.00	02/15/2000	30000.1	Clements, John
	208	147	USD$10,806.00	02/15/2000	30000.2	Clements, John
	208	147	USD$10,806.00	02/15/2000	30000.3	Clements, John
	207	148	USD$3,455.00	02/15/2000	90109.1	Trickey, Gina O.
	201	149	USD$10,485.00	02/15/2000	10901.1	Trickey, Gina O.
	201	149	USD$10,485.00	02/15/2000	10901.2	Trickey, Gina O.
	201	149	USD$10,485.00	02/15/2000	10901.3	Trickey, Gina O.

Sample Visual Component

Developers invoke visual components with uniform resource locators (**URLs**). For example, to invoke the Prebill Review component, use this URL:

https://<server_name>/CMSNet/billing/prebillreview/prebillreview.aspx

…where server_name is substituted with your server name.

Note: https indicates a SSL (Secure Socket Layer) connection. See the application-specific API Reference for URLs of specific visual components.

Security

Users must be granted execute rights on a VC in order to access it. In addition, the user must also be granted rights to any BLC functionality invoked by the VC. Functionality that cannot be accessed by the user due to security restrictions will typically be rendered as disabled.

Parameters

The manner in which a visual component behaves can be affected by sending additional parameters on its URL. Parameters can be placed in any order on the URL; however, care must be taken to properly escape any special characters in values that are not permitted by industry standards such as the question mark (?) and ampersand (&).

Most list-based visual components accept a filter parameter that allows the caller to restrict the range of rows being returned by supplying an entity filter expression. See Entity Filter Expressions for more details

The parameters and possible values available for each visual component are found in the application-specific API reference documents.

Entity Filter Expressions

Both business logic components and visual components utilize entity filter expressions to indicate which entity instances should be considered. These expressions allow complex filtering criteria to be expressed on any fields defined by the entity.

DAC Filters

Developers can restrict the rows returned by a DAC's **Read** method by supplying an appropriate filter expression instead of passing null or an empty string as the filter parameter.

Visual Component List Filters

Developers can narrow the data content of some visual components by providing filter criteria in the visual component's URL. Any VC that accepts filters will have a parameter called **userFilter** (or something similar) that accepts an entity filter expression. Filters added to the visual component URL append additional WHERE clause information to the underlying SQL query.

A simple filter could include an timekeeper employee code of “591”.

http://<servername>/CMSNet/billing/timedetailList/TimeDetailList.aspx?mode=Review&userFilter=(TimeEntry.TKEmpl.Code is 591)

Important: The maximum QueryString size for a URL is 2083 bytes, which limits the size of the filters that developers can include.

Expression Syntax

Passing filter values though the API requires the adherence to a number of standards. The standards are based on the fact that the application server is providing access to users across the enterprise without knowledge of their international settings.

- **Date Format** – Date values must be passed in **YYYY-MM-DD** format.
- **Decimal Character** - All numeric fields must use the period (.) character as the decimal identifier.
- **No Thousands Separator**
- **Negative Numbers** - The hyphen (-) character preceding numeric values denotes negative numbers.
- **No Currency Symbol**

Filter values must be Query by Example (QBE) expressions. QBE expressions support the meta-characters listed in the following table. Visual component filters concatenate single field QBE expressions across a number of fields.

General Syntax	**<filter>** ::= { <*fieldExpr*> } [{AND | OR } …]
Note: Every *fieldExpr* must be immediately surrounded by parentheses.	**<fieldExpr>** ::= { (<*fieldPath*> IS <*qbeExpr*>) }
	<fieldPath> ::= a period-delimited path to the field to be filtered upon from the root of the entity
	<qbeExpr> ::= a valid QBE expression given the data type of <*fieldPath*>
Examples	**Simple Filter:** (TimeEntry.TKEmpl.Code is 591)
	Complex Filter: **(**(TimeEntry.TKEmpl.Code is 100) or (TimeEntry.Dept is 101))

Important: Do not use the comma as an “OR” operator within a single QBE search expression, unless the filter value is a string. Strings can contain “OR” operators or commas, if desired. For non-string values, use the semicolon to represent “OR” for multiple-clause QBE expressions.

QBE Meta-Characters

The following table shows meta-characters for QBE expressions. Other characters found in QBE expressions are treated as an exact match requirement.

Group	Meta-characters	Notes
Wildcard Expressions	* ?	* substitutes for zero or more characters and maps directly to the SQL percent (%) wildcard. ? matches a single character at the position in the string at which the symbol is inserted and maps to the SQL underscore for matching a single character.
Space	@	Spaces are stripped out of QBE expressions. Use this symbol as a placeholder to match a space.
Variable	%	The %, prefixing a pre-defined constant, is replaced with the corresponding user or specific string value to which the variable is mapped.
Ranges	< > = !	In addition to the keyword 'TO,' these characters are used to indicate a range of values. The 'TO' keyword cannot appear in an expression with other range characters. With the exception of the 'TO' keyword, not all range characters are valid for all data types. The < > and = are used in regular expressions. The ! is used to denote negation (read as 'not') and will be processed as on and off as each ! is encountered in a single expression.
Composites	;	An expression delimiter, the semicolon expresses an 'OR' operator in the resultant SQL predicate. The semicolon must be used for all value types except strings, for which either semicolons or commas can be used.
Simple Arithmetic	PLUS MINUS	These characters are used to recursively perform addition or subtraction on the specified operands the end of the expression. QBE arithmetic is not valid for string data types.

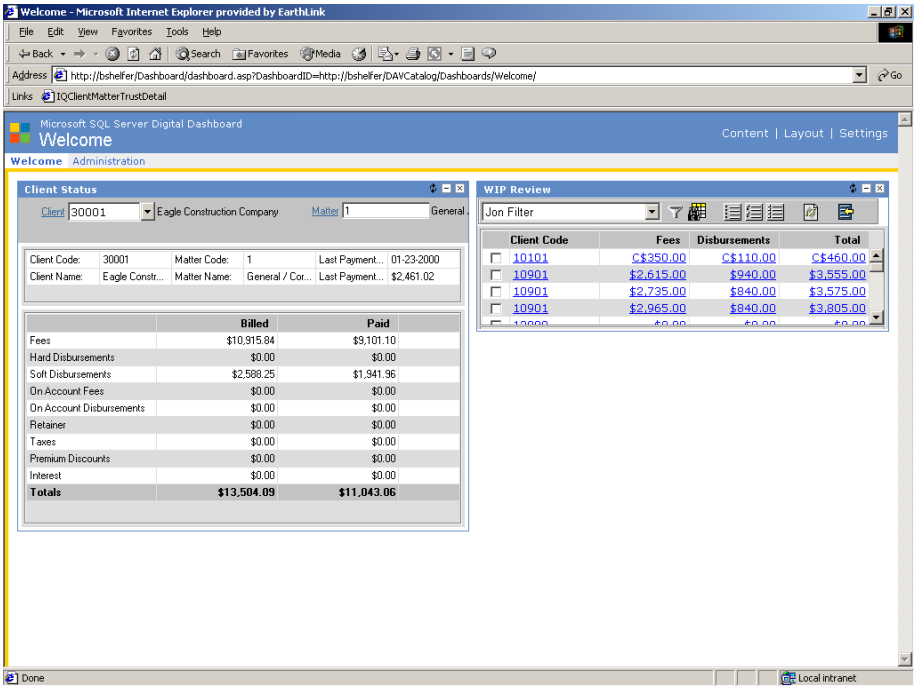
Consuming Visual Components

Execution Environments

Visual components provide developers with the opportunity to integrate and deploy portions of Expert Back Office functionality in custom solutions. These include portals, email systems, web applications, and other browser control-capable environments.

Portals

Visual components are well suited for incorporation into portal environments. Support for automatic resizing, login and QBE filtering makes it easy for seamless integration. By simply specifying the visual component's URL, the appropriate parameters and optional filter, developers can push information to the users.

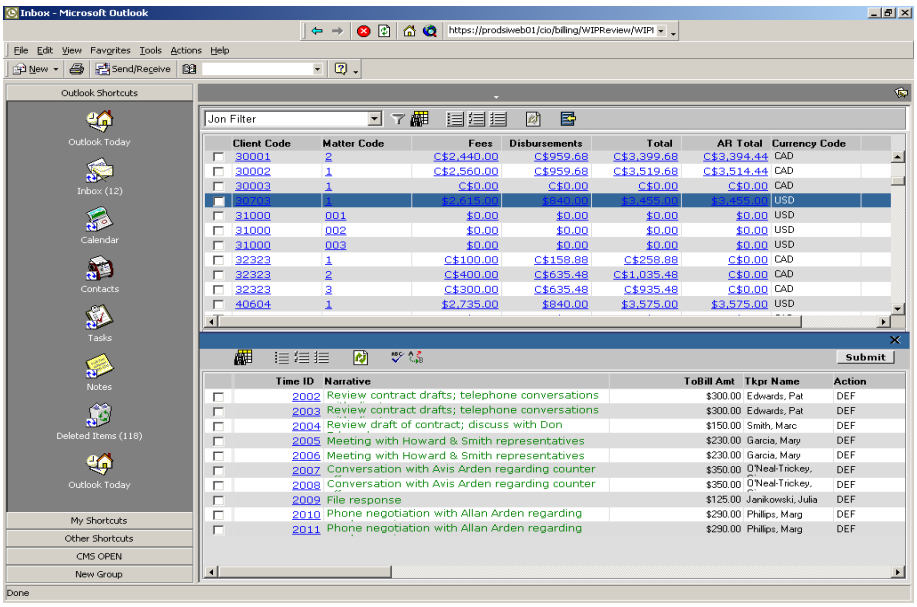


Visual Components in MS SharePoint

Email Systems

Many of today's email systems provide support for the hosting of the Internet Explorer browser control. Utilizing this capability, users can set up folders to display visual components embedded in their email client.

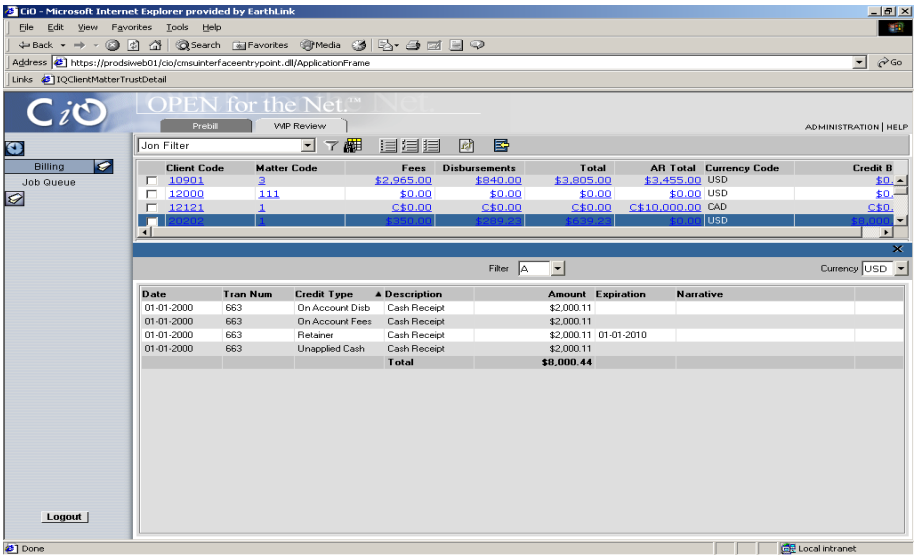
The figure below shows the WIP review component deployed as a folder inside of Microsoft's Outlook email client. Setting this up was a simple matter of specifying the URL for the component as the folder's home page property.



Visual Components in MS Outlook

Web Applications

The Expert Back Office suite of modules is built entirely on the concept of visual components. The components are consumed by the menu infrastructure of Expert Back Office. Visual components themselves are also consumed by other visual components. Developers can utilize visual components within their own web applications by providing the component with a host FRAME or IFRAME.



Visual Components in CMS.Net

Browser Control Environment

In order to deploy visual components in custom applications, developers must place them in a Microsoft **Internet Explorer 6** browser control. Earlier versions of Internet Explorer (IE) are not compatible with Expert Back Office visual components. This VB script example shows the code required to deploy a visual component in an IE6 browser control.

```
Option Explicit
Private Sub IQClientSummary_Click()
    ie6control.Navigate_
("<server_name>/CMSNet/inquiries/iqclientsummary/iqclientssummary.asp
x ")
End Sub
Private Sub WIPReview Click()
    ie6control.Navigate_
("<server_name>/CMSNet/billing/wipreview/wipreview.aspx ")
End Sub
```

Considerations

Host Environment

No special hosting environment is needed to access visual components. The same web servers installed using the *Web Application Installation* can be used to execute both the Expert Back Office browser applications and invoke visual components.

Authentication

If the host environment does not authenticate the user using the authentication API, then a visual component will attempt to authenticate the user, redirecting first to the login form if necessary. For portal use, it is highly recommended that Windows authentication be enabled.

Consuming BLCs

Execution Interfaces

Expert Back Office provides both a native .NET interface as well as a web services interface to execute methods of business logic components. Both have strengths and weaknesses that make them appropriate under different circumstances.

.NET Interface

The business logic component methods can be accessed directly using .NET by including the assemblies containing the components as references or dynamically loading them. The components can also be executed on a different host through .NET remoting,

This is the fastest interface to the functionality exposed by Expert Back Office. To call .NET components directly, you must use an integrated development environment (IDE) that supports the .NET Framework.

About the BLC Assemblies

Implementation

The BLC assemblies are imnplemented in .NET as private assemblies and thus, do not reside in the .NET global assembly cache (GAC). All supporting assemblies must exist in the same directory as the executing process (unless explicitly loaded dynamically from a specific place). Failure to do so will result in a .NET runtime exception.

Copying the BLC Assemblies

The assemblies containing the business logic components you wish to access can be copied from the installation target and deployed elsewhere. However, you must provide a *pointer.cfg* file in the executing directory that associates them with a specific installation's configuration file.

Any other assemblies that they directly reference must also reside in this execution directory. This would include ToolsBase.dll and ToolsCOM.dll, but each application-specific assembly has other direct references.

Note: If you elect to copy the BLC assemblies elsewhere, be aware that the Expert Back Office installation wizard will not know this and will not automatically upgrade them when applying an update.

Upgrading CMS.Net

After upgrading to a new version of CMS.Net, any .NET code containing an explicit reference to the BLC assemblies will need to be relinked.

Web Services Interface

To use the web services interface requires an environment capable of making SOAP calls over hypertext transfer protocol (HTTP). For instance, JavaScript in Internet Explorer (IE) and .NET capable languages meet this criterion.

Some .NET-capable languages include (but are not limited to):

- Microsoft Visual Basic .NET
- Microsoft Visual C# .NET
- Microsoft Visual C++ .NET

Authentication

Use of the web services interface requires that the authentication web service be used first to authenticate the user. Using the .NET interface to gain authentication will not be recognized by the web services interface.

Extending Functionality

BLC Events

BLCs publish events that are fired before and after a BLC updates the database during each call to a BLC's Create, Update, or Delete method. The events are published using two different mechanisms:

- **.NET Events:** .NET events allow developers to intercept events occuring on specific instances of a BLC. Each event has its own delegate that roughly mimics the parameters to the BLC method that fires it. The delegate for these events is BLC-dependent due to differing arguments passed to the delegate.
- **COM+ Events:** COM+ Events allow developers to intercept events fired by any .NET BLC instance, regardless of what process created the BLC instance. Under COM+ Events, developers can implement an event interface published by the BLC. This event interface contains methods prototypes that mirror the individual .NET events on the BLC. Once developers have created an object that implements the BLC's event interface, they can create a subscription in the COM+ catalog that links their object with the BLC's event class, which allows COM+ to instantiate the custom object and send it events whenever a BLC of that type is used to update the database.

If the custom event handling code determines that processing should be stopped and the transaction should be aborted, it can throw an exception to cause the transaction to be rolled back, or vote to abort the transaction on its own.

Important: Each .NET BLC typically has a web service equivalent. Which type of BLC you choose depends upon the needs of your environment. In many cases, the methods and parameters for the .NET BLC and its web service equivalent are exactly the same. See individual topics in the Reference section for more information.

About Return Messages: The **Create** function of each .NET BLC returns a value when it successfully executes. The **Update** and **Delete** functions, however, do not confirm success. All three functions will generate an exception if a problem occurs during execution.

For additional information about this topic, see the knowledge base article K502441, "Extending CMS.Net API Transactions using COM+ Events."

Considerations

Host Environment

No special hosting environment is needed to access the BLC components. The same web servers installed using the *Web Application Installation* can be used to execute the Expert Back Office browser applications and invoke business logic components (through their web services interface).

Authentication

BLC components require that the user be authenticated. If the host environment does not authenticate the user using the authentication API, then the BLC will throw an exception when a method is accessed.

OfficeXtension

Authentication Options

The **options** parameter of the Login method contains a **UseOfficeXtension** property that corresponds to the "Use OfficeXtension" checkbox displayed on the login form (when in an OfficeXtension deployment). When this property is true, the authenticated user's satellite database will be used (when applicable) for database connections. If false, the central database will always be used.

For more details, see the **Login** method described in the Authentication section of the API Reference.

Specifying a Target Data Source

The DbLock, OwnerlessDbLock, and BOS RequestManager BLCs have no way to determine what data source to target when used in an OfficeXtension deployment. It is up to the code that is consuming these BLCs to specify the target data source. The .NET and web service interfaces each have their own way of doing this.

.NET

When the .NET versions are used in an OfficeXtension deployment, the consuming code is responsible for the following:

- Getting the current datasource via CioPrincipal.Current.GetDatasource() and saving it for use after the BLC method is called
- Setting the desired Datasource via CioPrincipal.Current.SetDatasource() before calling the BLC method
- Restoring the saved Datasource via CioPrincipal.Current.SetDatasource() after calling the BLC method

For instance, here is an example of calling DbLock's Unlock method in an OfficeXtension environment:

Example code to dynamically set the datasource target of a requested lock.

```
ICioPrincipal cp = CioPrincipal.Current;
DataSource previousDS = cp.GetDatasource();
try
{
   // Target the desired datasource context
   cp.SetDatasource(OfficeXtensionBehavior.Central);
   using (DbLock lockBLC = new DbLock())
   {
         lockBLC.Unlock();
   }
}
finally
{
   // Use a finally block.
   // This ensures the context will always get restored.
   cp.SetDatasource(previousDS);
}
```

For information about the GetDatasource and SetDatasource methods, see the Data Source Targeting API subsection below.

Web Services

When the web service versions are used, the process is simple. Each web service method that requires an OfficeXtension Behavior in order to determine its datasource target will have an OXBehavior argument. The consuming code must simply pass the Behavior as text in that argument (for instance, “Central”, “Satellite”, etc.). For a list of possible OfficeXtension behaviors, see the OfficeXtensionBehavior enumeration below.

Getting Started

This section provides examples of calling BLCs using the various interfaces and authentication methods. Included are:

- Calling a BOS-based BLC using the web services platform and SOAP headers for authentication.
- Invoking a BLC locally using the .NET authentication class.
- Invoking a BLC remotely using the web services platform, supplying a username/password.
- Invoking a BLC remotely using the web services platform, supplying a Windows NTLM credential.

Example 1

The following Visual Basic 6.0 code snippet outlines the call of a billing BLC to draft bills for two specific prebills identified by tran_uno codes of **1025** and **1027**.

Please note that **CioHeaderHandler.cls** file is available in the \SampleCode directory.

```
Option Explicit
Option Base 0

' These constants describe the priority at which a BOS job should run
Private Const PRIORITY_HIGH As Integer = 0
Private Const PRIORITY_NORMAL As Integer = 1
Private Const PRIORITY_LOW As Integer = 2

' These constants describe what to do with locks when the BOS request
completes
Private Const LOCKS_RELEASE As Integer = 0
Private Const LOCKS_TRANSFER As Integer = 1

' Location of web services - <server name> must be replaced with the actual
name
' of the server hosting the web services
Private Const webServiceBaseURL = “<server name>/CMSNet/Tools/ToolsWS/”

' Billing draft bill parameters
Private Const DraftBillParams = “<jobParams type=””BILLING””
action=””DB””><parameters tranUnoList=””1025,1027””></parameters></jobParams>”

' Create a custom header handler to work with the SOAP headers required for
' authentication
Dim headerHandler As New CioHeaderHandler

' Create SoapClient instances for each web service
Dim authenticationService As New MSSOAPLib.SoapClient
Dim functionalSecurityService As New MSSOAPLib.SoapClient
Dim dbLockService As New MSSOAPLib.SoapClient
```

```
Dim bosService As New MSSOAPLib.SoapClient

' Put the header handler in place in each SoapClient
Set authenticationService.headerHandler = headerHandler
Set functionalSecurityService.headerHandler = headerHandler
Set dbLockService.headerHandler = headerHandler
Set bosService.headerHandler = headerHandler

' Initialize the SoapClients with the WSDL descriptions of the web services
authenticationService.mssoapinit webServiceBaseURL &_
"AuthenticationService.asmx?WSDL"
functionalSecurityService.mssoapinit webServiceBaseURL &_
"FunctionalSecurityService.asmx?WSDL"
dbLockService.mssoapinit webServiceBaseURL & "DbLockService.asmx?WSDL"
bosService.mssoapinit webServiceBaseURL & "BOSService.asmx?WSDL"

' Login into CMS.Net - must be a valid login and password into CMS.Net for
Client/Server
Dim success As Boolean
success = authenticationService.Login("CMSUSER", "123")

' Check to make sure the current user can perform the draft bill task we're
' requesting the module id details are found in the reference for the BLCs
' success = functionalSecurityService.UserHasRights("1,50000,50300,50310",
' "Execute")
' Lock the appropriate rows for the task
Dim lockList(1) As Integer
lockList(0) = 1025
lockList(1) = 1027
success = dbLockService.Lock("Lock created from VB app", "BLT_BILL", lockList)

' Create the request in the queue
Dim jobUno As Integer
jobUno = bosService.CreateRequest("BILLING", "VB Bill Draft Request",_
DraftBillParams, "VBAPP", PRIORITY_NORMAL)

' Transfer all the locks that belong to us (just one) to the BOS request
dbLockService.TransferAll "BOS", jobUno

' Allow BOS to process the request
bosService.SubmitRequest jobUno, LOCKS_RELEASE

' Get information about the job (see BOSService.asmx?schema=RequestInfo for
more
' information)
Dim requestInfo As MSXML2.IXMLDOMNodeList
Set requestInfo = bosService.GetRequestByJobUno(jobUno)
Dim schema As MSXML2.IXMLDOMElement
Set schema = requestInfo(0)
Dim requests As MSXML2.IXMLDOMElement
Set requests = requestInfo(1)
Dim statusElm As MSXML2.IXMLDOMElement
Set statusElm = requests.selectSingleNode_
("//RequestInfo/BOS_REQUEST_QUEUE[0]/STATUS")
MsgBox "Job number " & jobUno & " currently has a status of '" &
statusElm.Text_ & "'.", vbOKOnly, "BOS Queue Result"
```

```
' Logout of CMS.Net
authenticationService.Logout
```

Example 2

The following C# code sample uses the .NET platform to access the components directly by supplying a username/password to the authentication BLC. It retrieves client data from the database based on a client code, changes the jurisdiction code of the client, and creates an alias for the client. To authenticate based on a login authority such as Windows or NetWare, use the other method overload for Login().

```
static void Main(string[] args)
{
   IPrincipal previousPrincipal = null;

   try
   {
         // First, log into the CMS.Net system. The Login method returns the
previous
         // IPrincipal reference associated with the thread.
         string Username = "Username";
         string Password = "Password";
         previousPrincipal = Authentication.Login(Username, Password);

         // Perform a lookup on client data based on a client code.
         ClientData cd;
         using (ClientDAC clientDAC = new ClientDAC())
         {
               cd = clientDAC.Read("(Client.Code is 10901)")[0];
         }

         // Change the client's jurisdiction code.
         ClientDataUpdate cdu = cd.GetDataUpdate();
         cdu.JurisdicCode = "NA";
         using (ClientBLC clientBLC = new ClientBLC())
         {
               clientBLC.Update(cd.ClientUno, cdu);
         }

         // Create a new name alias for the client.
         NameAliasDataCreate nadc;
         using (NameAliasDAC nameAliasDAC = new NameAliasDAC())
         {
               // Get a blank NameAliasDataCreate reference initialized with
some defaults.
               nadc = nameAliasDAC.Initialize().GetDataCreate();
         }

         // Add our information
         nadc.NameUno = cd.NameUno;
         nadc.Name = "Alias for Client 10901";
         nadc.EffectiveDate = DateTime.Now;
         nadc.Relationship = "AKA";
         using (NameAliasBLC nameAliasBLC = new NameAliasBLC())
         {
               int aliasUno = nameAliasBLC.Create(nadc);
```

```
        }
    }
    finally
    {
        // Make sure we log out of CMS.Net if an error occurs.
        Authentication.Logout(previousPrincipal);
    }
}
```

Example 3

The following C# code sample utilizes the web services platform passing a supplied username/password to the authentication web service. Like the other examples, it retrieves client data from the database based on a client code, changes the jurisdiction code of the client, and creates an alias for the client.

```
// This example assumes the URL used to access the web services in the
// CMS.Net API is using "Forms Authentication."
using System;

namespace WindowsAuthAPITest
{
    class FormsAuthWSExample
    {
        [STAThread]
        static void Main(string[] args)
        {
            string Username = "username";
            string Password = "password";

            // First, log into the CMS.Net system.
            AuthenticationService.AuthenticationService auth = new
AuthenticationService.AuthenticationService();
            auth.CookieContainer = new System.Net.CookieContainer();

            bool loggedIn = auth.Login(Username, Password, null);

            // Perform a lookup on client data based on a client code.
            ClientService.ClientService cs = new
ClientService.ClientService();
            cs.CookieContainer = auth.CookieContainer;
            ClientService.ClientData cd = cs.Read("(Client.Code is
10901)")[0];

            // Change the client's jurisdiction code.
            ClientService.ClientDataUpdate cdu = cs.GetDataUpdate(cd);
            cdu.JurisdicCode = "NA";
            cs.Update(cd.ClientUno, cdu);

            // Create a new name alias for the client.
            NameAliasService.NameAliasService nas = new
NameAliasService.NameAliasService();
            nas.CookieContainer = auth.CookieContainer;

            NameAliasService.NameAliasDataCreate nadc =
nas.InitializeDataCreate();
            nadc.NameUno = cd.NameUno;
```

```
                nadc.Name = "Alias for Client 10901";
                nadc.EffectiveDate = DateTime.Now;
                nadc.Relationship = "AKA";

                int aliasUno = nas.Create(nadc);

                // Logout.
                auth.Logout();
            }
    }
}
```

Example 4

The following C# code sample utilizes the web services platform using Windows authentication by passing a NTLM credential. Like the other examples, it retrieves client data from the database based on a client code, changes the jurisdiction code of the client, and creates an alias for the client.

```
// This example assumes the URL used to access the web services in
the
// CMS.Net API is using "Windows Authentication."
using System;

namespace WindowsAuthAPITest
{
   class WindowsAuthWSExample
   {
         [STAThread]
         static void Main(string[] args)
         {
               // First, log into the CMS.Net system.
               ClientService.ClientService cs = new
ClientService.ClientService();
               cs.CookieContainer = new
System.Net.CookieContainer();

               // Depending on whether you want to specify the
credentials used to connect to
               // the web server, use one of the following:
               bool useCurrentUsersCredentials = true;
               if (useCurrentUsersCredentials)
               {
                     cs.Credentials =
System.Net.CredentialCache.DefaultCredentials;
               }
               else // Use credentials specified by the developer.
               {
                     System.Net.CredentialCache cc = new
System.Net.CredentialCache();
                     cc.Add(new
System.Uri("http://www.mycmsnetserver.com/"), "NTLM",
                    new System.Net.NetworkCredential("username",
"password", "domain"));
                     cs.Credentials = cc;
               }
```

```
                // Perform a lookup on client data based on a client
code.
                ClientService.ClientData cd = cs.Read("(Client.Code
is 10901)")[0];

                // Change the client's jurisdiction code.
                ClientService.ClientDataUpdate cdu =
cs.GetDataUpdate(cd);
                cdu.JurisdicCode = "NA";
                cs.Update(cd.ClientUno, cdu);

                // Create a new name alias for the client.
                NameAliasService.NameAliasService nas = new
NameAliasService.NameAliasService();
                nas.CookieContainer = cs.CookieContainer;
                nas.Credentials = cs.Credentials;

                NameAliasService.NameAliasDataCreate nadc =
nas.InitializeDataCreate();
                nadc.NameUno = cd.NameUno;
                nadc.Name = "Alias for Client 10901";
                nadc.EffectiveDate = DateTime.Now;
                nadc.Relationship = "AKA";

                int aliasUno = nas.Create(nadc);

                // Logout.
                AuthenticationService.AuthenticationService auth =
new AuthenticationService.AuthenticationService();
                auth.CookieContainer = cs.CookieContainer;
                auth.Credentials = cs.Credentials;
                auth.Logout();
            }
    }
}
```

API Reference

Authentication

The authentication mechanism used to validate user credentials differs depending upon whether you are using .NET BLCs or web services. Please note that interface you call determines the type of BLC you must use to call other functions. For instance, if you authenticate the user via AuthenticationService, then you must also check the user's rights via a web service, as well. If you authenticate the user via the Authentication .NET component, however, you must then perform other functions via .NET components.

The Authentication BLC provides functions to log into and out of the Expert Back Office system when using .NET BLCs directly. This BLC works by replacing the underlying .NET "principal."

The AuthenticationService web service provides this same functionality for users of web services. This BLC validates the incoming user credentials and issues cookies and SOAP headers, one of which must be passed back the web server during subsequent web service calls.

When the AuthenticationService's Login method is called, it will issue both cookies and SOAP headers containing the information required to re-authenticate the user for other web service method calls. Web service consumers written in IE 6 or a .NET language can use cookies and ignore the SOAP headers. Web service consumers working through the Microsoft SOAP Toolkit 2.0 can ignore the cookies and work directly with the SOAP headers passed back. Either cookies or SOAP headers *must* be supplied with each call to an Expert Back Office web service.

Authentication/AuthenticationService.asmx

Location

Assembly	Namespace
ToolsBase.dll	Solution6.Cio.Tools

Methods

These methods apply to equivalent BLCs for both the .NET and web services platforms.

For AuthenticationService (web service) and Authentication (.NET) BLCs:

LoggedIn

```
bool LoggedIn ()
```

Returns *true* if the current user has been logged into CMS.Net.

Login

```
.NET: IPrincipal Login (string username, string password[,
LoginOptions options])
.NET: IPrincipal Login ([LoginOptions options])
WS: bool Login (string username, string password,
LoginOptions options)
```

The .NET versions replace the principal reference on the current thread with a custom principal object. The current thread's previous principal is returned, and can be passed into the Logout method to restore the original principal, if desired.

The second .NET method overload authenticates the current user based on a login authority instead of requiring a username and password to be supplied.

Omitting the options parameter from either .NET method results in the classes default property values being used.

The web service version of this method returns true if the user could be validated, or false if not successful. It sets a cookie and SOAP headers in the response in the same way that the web application login form does. Either this cookie or the SOAP headers must be passed by the caller to any secured Expert Back Office API methods. This cookie and SOAP headers do not obey the persistent login feature of Expert Back Office and are non-durable.

Supporting Types:

```
public class LoginOptions
{
   public LoginAuthorityType LoginAuthority { get; set; }
   public string ModuleName { get; set; }
   public bool UseOfficeXtension { get; set; }
}
```

The LoginAuthority property determines the mechanism used to determine the user's identity. It defaults to AutoDetect, which tries to authenticate using a Microsoft Networks provider. If not available, it will try to authenticate using a Novell NetWare provider. To authenticate using only a specific login authority, use a value of either Windows or NetWare.

The ModuleName property sets the module name used for locking. Unless you have a specific need, it can remain unset.

The UseOfficeXtension property only has meaning in an OfficeXtension environment. For information about this property, see the OfficeXtension Considerations section.

```
public enum LoginAuthorityType
{
   None,
   AutoDetect,
   Windows,          // Microsoft Networks provider
   NetWare           // Novell NetWare provider
}
```

Logout

```
.NET: void Logout (IPrincipal previousPrincipal)
WS: void Logout ()
```

Note: Logs the caller out of CMS.Net. Future calls to secured BLCs will generate an exception until the Login function is called again.

The .NET version takes an optional IPrincipal parameter. If specified, the principal on the current thread will be reset to the IPrincipal reference passed in.

UserHasRights

```
.NET: bool UserHasRights (string modulePath, string rightsName)
```

Returns true if the current user has the rights specified. See the User Rights description for more information on the parameters passed into this method.

This is a .NET-only method. Use the Functional Security Service for its Web Service equivalent.

WhoAmI

```
string WhoAmI ()
```

Returns the username that was used to log into the Expert Back Office system.

CioPrincipal

Each thread executing in .NET has an associated principal object instance that represents the security context under which code is running. In Expert Back Office, the principal is used to:

- Keep track of the current user context.
- Enforce security.
- Target the default datasource.

The CioPrincipal class provides a convenient way of obtaining the current Expert Back Office principal instance. It is only available from the .NET interface.

Location

Assembly	Namespace
ToolsBase.dll	Solution6.Cio.Tools

Methods

CioPrincipal

```
static ICioPrincipal Current
```

Returns an interface refererence to the current Expert Back Office principal associated with the thread. Expert Back Office uses different principal classes for web and Windows applications; however, they both support a common interface, ICioPrincipal.

ICioPrincipal

The ICioPrincipal interface is implemented by all Expert Back Office principal classes. These interface methods can be invoked using the interface reference obtained from the CioPrincipal.Current method described above. It is only available from the .NET interface,

Location

Assembly	Namespace
ToolsBase.dll	Solution6.Cio.Tools

Methods

GetDataSource

```
InternalDataSource GetDataSource()
```

Returns a pointer the current user's default data source context.

SetDataSource

```
void SetDataSource(InternalDataSource datasource)
```

Sets the user's default data source context based on a supplied internal datasource reference.

SetDataSource

```
void SetDataSource(OfficeXtensionBehavior OXBehavior)
```

Sets the user's default data source according the specified OfficeXtension behavior.

Supporting Types:

```
public enum OfficeXtensionBehavior
{
   Central,
   Satellite,
   Closest,
   None,       // Should not be used
   Dynamic     // Should not be used
}
```

BOS Request Manager

The Business Object Server (BOS) handles incoming requests for distributed processing. To do so, BOS invokes BLCs on the BOS server, instead of on the web server or on the workstation.

BOSService is a web service BLC that creates job requests and manages them in the database queue. The RequestManager is the .NET equivalent of BOSService.

RequestManager/BOSService.asmx

Location

Assembly	Namespace
ToolsCOM.dll	Solution6.Cio.Tools

Methods

These methods apply to equivalent BLCs for both the .NET and web services platforms.

For BOSService (web service) and RequestManager (.NET) BLCs:

CreateRequest

```
.NET: int CreateRequest (string requestType, string
description, string jobParams, string context[,
RequestPriorities priority])
WS: int CreateRequest (string requestType, string description,
string jobParams, string context, int priority, string
OXBehavior)
```

Creates a job in the request queue. Once created, the job will not be processed by a business object server until SubmitRequest or SubmitRequestSync is called. The format of the *jobParams* string is specific to the BLC that will be invoked. The return value is the JOB_UNO of the request that was created. The *context* parameter can be used later to filter out requests in the queue that came from a specific application.

In the .NET version of the BLC, the *priority* parameter is specified using the RequestPriorities enumeration (**High**, **Normal**, and **Low**).

In the web service version *priority* can be a value from **0** to **2**, with **0** equating to **High**, 1 equating to **Normal**, and 2 equating to **Low**.

The web service version also requires an additional OXBehavior argument. This parameter must be null unless this method is being used in an OfficeXtension environment (see the OfficeXtension Considerations section).

Supporting Types:

```
public enum RequestPriorities
{
   High,
   Normal,
   Low
}
```

SubmitRequest

```
.NET: void SubmitRequest (int jobUno[, LockingOptions
lockOptions])
WS: void SubmitRequest (int jobUno, int lockOptions, string
OXBehavior)
```

Asynchronously submits a job created using CreateRequest for processing. This method simply updates the request to allow it to be processed, and returns immediately.

In the .NET version of this BLC, the *lockOptions* parameter is specified using the LockingOptions enumeration. Specifying **Release** causes all locks transferred to the BOS row to be released upon job completion, while **Transfer** will cause the locks to be transferred back to the original web session owner.

In the web service version *lockOptions* can be a value of **0** or **1**, with **0** equating to **Release** and **1** equating to **Transfer**.

The web service version also requires an additional OXBehavior argument. This parameter must be null unless this method is being used in an OfficeXtension environment (see the OfficeXtension Considerations section).

Supporting Types:

```
public enum LockingOptions
{
   Release,    // drop the locks when the job is completed
   Transfer    // return the locks back to the originator
}
```

SubmitRequestSync

```
.NET: RequestInfo SubmitRequestSync (int jobUno[,
LockingOptions lockOptions])
WS: RequestInfo SubmitRequestSync (int jobUno, int
lockOptions, string OXBehavior)
```

Synchronously submits a job created using CreateRequest for processing. This method updates the request to allow it to be processed, waits until processing completes, and returns information from the BOS_REQUEST_QUEUE table regarding the job.

In the .NET version of this BLC, the *lockOptions* parameter is specified using the LockingOptions enumeration (**Release** and **Transfer**). Specifying **Release** causes all locks transferred to the BOS row to be released upon job completion, while **Transfer** will cause the locks to be transferred back to the original web session owner.

In the web service version *lockOptions* can be a value of **0** or **1**, with **0** equating to **Release** and **1** equating to **Transfer**. The web service version also requires an additional OXBehavior argument.

The web service version also requires an additional OXBehavior argument. This parameter must be null unless this method is being used in an OfficeXtension environment (see the OfficeXtension Considerations section).

Supporting Types:

```
public enum LockingOptions
{
   Release,    // drop the locks when the job is completed
   Transfer    // return the locks back to the originator
```

```
}
```

ChangeRequestStatus

```
.NET: public void ChangeRequestStatus (int jobUno, string
newStatus)
WS: public void ChangeRequestStatus (int jobUno, string
newStatus, string OXBehavior)
```

Allows applications to change the status of a request. For example, this function allows jobs to be deleted at certain points in their lifecycle. Throws an exception if the requested status is not valid given the job's current status, or if the job cannot be changed (usually because processing has started on it).

These status changes are allowed:

- Change created (**C**) to delete (**L**).
- Change queued (**Q**) to hold (**H**) or delete (**L**).
- Change hold (**H**) to queued (**Q**) or delete (**L**).
- Change Done (**D**) to Keep (**K**) or delete (**L**).
- Change Keep (**K**) to delete (**L**).
- Change error (**E**) to delete (**L**).

The web service version also requires an additional OXBehavior argument. This parameter must be null unless this method is being used in an OfficeXtension environment (see the OfficeXtension Considerations section).

GetRequests

```
.NET: RequestInfo GetRequests (string filter)
WS: RequestInfo GetRequests (string filter, string
OXBehavior)
```

This is a general-purpose function to return requests based on a set of filter criteria. The BOS Request entity is used to create filters for the GetRequests method.

The web service version also requires an additional OXBehavior argument. This parameter must be null unless this method is being used in an OfficeXtension environment (see the OfficeXtension Considerations section).

GetRequestByJobUno

```
.NET: RequestInfo GetRequestByJobUno (int jobUno)
WS: RequestInfo GetRequestByJobUno (int jobUno, string
OXBehavior)
```

This function takes a *jobUno* and returns information about the job. Note that if the requested *jobUno* does not exist, no exception is thrown.

The web service version also requires an additional OXBehavior argument. This parameter must be null unless this method is being used in an OfficeXtension environment (see the OfficeXtension Considerations section).

GetRequestByContext

```
.NET: RequestInfo GetRequestsByContext (string context)
WS: RequestInfo GetRequestsByContext (string context,
string OXBehavior)
```

This function takes context string and returns all the jobs for that context. The list is automatically filtered for the current user and also filters out jobs in the "delete" or "deleting" statuses.

The web service version also requires an additional OXBehavior argument. This parameter must be null unless this method is being used in an OfficeXtension environment (see the OfficeXtension Considerations section).

Database Locking

The DbLock and OwnerlessDbLock BLCs allow consumers to lock and unlock rows in the database. Many BLC methods require that certain database locks be held by the caller in order to execute them. Ownerless locks can be useful for preventing any user from obtaining specific locks through the DbLock BLC. An ownerless lock is only released by an explicit call to the unlock method. Ownerless locks can be established and broken by deferring individuals.

The DbLock BLC also allows consumers to transfer locks to other lock owners. There are two types of lock owners:

- ***Browser Session*** - Session ownership lasts for a set duration (default is 30 minutes) from the point at which they are created. They are automatically released if the session ends or the time expires.
- ***Business Object Server (BOS)*** - Occurs via lock transfer from a browser session and lasts while the BOS job is queued.

DbLock/DbLockService.asmx

Location

Assembly	Namespace
ToolsCOM.dll	Solution6.Cio.Tools

Methods

These methods apply to equivalent BLCs for both the .NET and web services platforms.

For DbLockService (web service) and DbLock (.NET) BLCs:

IsLocked

```
.NET: bool IsLocked (string lockName, int lockInstance)
WS: bool IsLocked (string lockName, int lockInstance, string
OXBehavior)
```

Returns *true* if the single lock specified is held by the currently logged in user.

The web service version also requires an additional OXBehavior argument. This parameter must be null unless this method is being used in an OfficeXtension environment (see the OfficeXtension Considerations section).

Lock

```
.NET: bool Lock (string description, string lockName, params int[]
lockInstances)
WS: bool Lock (string description, string lockName, int[]
lockInstances, string OXBehavior)
```

Attempts to get the specified array of exclusive locks for a specific *lockName*. A DbLockException is thrown if any of the requested locks could not be obtained.

The web service version also requires an additional OXBehavior argument. This parameter must be null unless this method is being used in an OfficeXtension environment (see the OfficeXtension Considerations section).

Unlock

```
.NET: void Unlock(string lockName, params int[] lockInstances)
WS: void Unlock(string lockName, int[] lockInstances, string
OXBehavior)
```

Releases the locks specified by *lockName* and instance array. If *lockInstances* is unspecified (null), then all owned instances of the specified *lockName* are unlocked.

The web service version also requires an additional OXBehavior argument. This parameter must be null unless this method is being used in an OfficeXtension environment (see the OfficeXtension Considerations section).

UnlockAll

```
.NET: void Unlock()
WS: void UnlockAll(string OXBehavior)
```

Releases all owned locks for the current owner.

The web service version also requires an additional OXBehavior argument. This parameter must be null unless this method is being used in an OfficeXtension environment (see the OfficeXtension Considerations section).

Transfer

```
.NET: void Transfer (LockOwner lockOwner, string lockName, params
int[] lockInstances)
WS: void Transfer (string lockOwnerType, string lockOwnerID, string
lockName, int[] lockInstances, string OXBehavior)
```

Transfers ownership of locks specified by *lockName* and instance array from the current owner to the *lockOwner* specified. If *lockInstances* is unspecified (null), then all owned instances of the specified *lockName* are transferred.

In the web service version of this BLC, the *lockOwnerType* should be “Session” or “Bos”, and the *lockOwnerID* a string representation of either an integer or Guid value.

The web service version also requires an additional OXBehavior argument. This parameter must be null unless this method is being used in an OfficeXtension environment (see the OfficeXtension Considerations section).

Supporting Types:

```
public enum LockOwnerTypes
{
   Bos,
   Session,
   Workstation
}
```

TransferAll

```
.NET: void Transfer (LockOwner lockOwner)
WS: void TransferAll (string lockOwnerType, string lockOwnerID,
string OXBehavior)
```

Transfers ownership of all locks from the current owner to the *lockOwner* specified.

In the web service version of this BLC, the *lockOwnerType* should be "Session" or "Bos", and the *lockOwnerID* a string representation of either an integer or Guid value.

The web service version also requires an additional OXBehavior argument. This parameter must be null unless this method is being used in an OfficeXtension environment (see the OfficeXtension Considerations section).

Supporting Types:

```
public enum LockOwnerTypes
{
   Bos,
   Session,
   Workstation
}
```

OwnerlessDbLock/OwnerlessDbLockService.asmx

Location

Assembly	Namespace
ToolsCOM.dll	Solution6.Cio.Tools

Methods

These methods apply to equivalent BLCs for both the .NET and web services platforms.

For OwnerlessDbLockService (web service) and OwnerlessDbLock (.NET) BLCs:

IsLocked

```
.NET: bool IsLocked (string lockName, int lockInstance)
WS: bool IsLocked (string lockName, int lockInstance, string
OXBehavior)
```

Returns *true* if the single lock specified exists.

The web service version also requires an additional OXBehavior argument. This parameter must be null unless this method is being used in an OfficeXtension environment (see the OfficeXtension Considerations section).

GetLockDescription

```
.NET: LockDescription GetLockDescription (string lockName,
int lockInstance)
WS: LockDescription GetLockDescription (string lockName, int
lockInstance, string OXBehavior)
```

Returns a LockDescription instance indicating the lock's reason, module, and creation date.

The web service version also requires an additional OXBehavior argument. This parameter must be null unless this method is being used in an OfficeXtension environment (see the OfficeXtension Considerations section).

Lock

```
.NET: bool Lock (string description, string lockName, params int[] lockInstances)
WS: bool Lock (string description, string lockName, int[] lockInstances, string OXBehavior)
```

Attempts to get the specified array of exclusive locks for a specific *lockName*. A DbLockException is thrown if any of the requested locks could not be obtained.

The web service version also requires an additional OXBehavior argument. This parameter must be null unless this method is being used in an OfficeXtension environment (see the OfficeXtension Considerations section).

Unlock

```
.NET: void Unlock(string lockName, params int[] lockInstances)
WS: void Unlock(string lockName, int[] lockInstances, string OXBehavior)
```

Releases the locks specified by *lockName* and instance array. If *lockInstances* is unspecified (null), then all instances of the specified *lockName* are unlocked.

The web service version also requires an additional OXBehavior argument. This parameter must be null unless this method is being used in an OfficeXtension environment (see the OfficeXtension Considerations section).

UnlockAll

```
.NET: void Unlock()
WS: void UnlockAll(string OXBehavior)
```

Releases all ownerless locks.

The web service version also requires an additional OXBehavior argument. This parameter must be null unless this method is being used in an OfficeXtension environment (see the OfficeXtension Considerations section).

Document Generation BLC

The Document Generation BLC generates documents based on the specified document template, the records specified, and the desired location for the resulting file. It allows developers to run custom DocuDraft startup files via the Business Object Server.

BOS Parameters

Request	Value
Request Type	DOCUDRAFT
<jobParams> Element	The root element must contain attributes containing the following values: type="DOCUDRAFT" action="DRAFT" version="43OP3"
<parameters> Element	The text of the <parameters> tag contains a DocuDraft startup file that should be executed. See the DocuDraft User Manual for more information on writing scripts

Request	Value
	for use by DocuDraft.

Note that the request type must also be redundantly passed to the **CreateRequest** method of the BOS Request Manager along with the XML text representing the job parameters.

Example

XML to generate a document using DocuDraft

```
<jobParams type="DOCUDRAFT" action="DRAFT"
version="43OP3">
   <parameters>
{DRAFT OFF}
{_DATE_INPUT = 'US'}
{_PATH_DRAFT = 'Q:\CMSOPEN4.3\DDTMPLT'}
{_PATH_IMAGE = 'X'}
{_PATH_WORK = 'Q:\CMSOPEN4.3\DDPBILL'}
{_EMPL_NAME = "SOL6"}
{_EMPL = 'SOL6'}
{_EMPL_INIT = 'SOL6'}
{_FIM_NAME="ABC Inc."
{VIEW DATABASE DD_FILEOPEN WHERE MATTER_UNO IN (549)
PRELOAD}
{CREATE DOCUMENT 'Q:\CMSOPEN4.3\DDFOPEN\' + 'ABC' + '.RTF'
USING 'Q:\CMSOPEN4.3\DDTMPLT\' + 'OPEN.rtf'}
{DRAFT ON}
{DRAFT 'Q:\CMSOPEN4.3\DDTMPLT\' + 'OPEN.RTF'}
{DRAFT OFF}
{END VIEW}
   </parameters>
</jobParams >
```

User Rights

The FunctionalSecurityService determines whether the currently logged in user has the appropriate rights to perform the requested function. With this service, developers can establish in advance whether the request for processing will fail due to security, and then prevent the user from making the request.

FunctionalSecurityService is available as a web service only. The **UserHasRights** method is available to .NET developers through the Authentication .NET BLC.

FunctionalSecurityService.asmx

Methods

FunctionalSecurityService implements a single function.

UserHasRights

```
bool UserHasRights (string modulePath, string rightsName)
```

Given a modulePath and rightsName, returns true if the current user can perform the specified action. To get the modulePath and rightsName, please see the "Security Considerations" section for the specific visual component or business logic component.

Getting Support

If you have any difficulty, we recommend that you obtain assistance from the following sources.

Support Portal

The Content tab of the Support Portal contains the following information:

- Release documents (Release Notes and Installation Guides).
- Product documentation (User, Administration, Customization, and Application Setup Guides).
- Knowledge Base articles relating to the installation, customization and use of the product.

To access these documents:

1. Access the Support Portal at *www.myaderant.com*

   Note: If you do not have login details for the Support Portal, please contact Aderant Support.

2. Click the **Content** tab.

   Tip: If the Content tab is not visible, click the unlabeled tab on the far right (only has a +), then click Content in the All Tabs list.

3. In the **Search in** field select **All Libraries** and enter words that may be in the title of the document.
4. Click **Go!**
5. Use the left pane to filter your results further. For example, you can filter by Installation Guide in the Content Type filter, or AE 8.1 in the Version filter.

User Help in Aderant Applications

Press *F1* in any application to launch the user help for the application.

Aderant Technical Help in Visual Studio

Administration and Customization help is available by clicking on the Aderant toolbar in Visual Studio.

Telephone Support

If the online help or our Support Portal do not answer your questions, Aderant Support representatives are available by phone.

Americas Region	+1 850 224 2004
Australia (within Australia)	1800 236 285
New Zealand (within NZ)	0800 849 000
Europe & Middle East	UK +44 (0) 20 7038 9696
	Netherlands +31 88 400 0300